

SECURITY AND GUARDRAILS

Module 4 — Governed AI Feature Delivery

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

-
-
-
-

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

- Prompt injection via user or document input
-
-
-

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

- Prompt injection via user or document input
- Sensitive data leakage risks
-
-

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

- Prompt injection via user or document input
- Sensitive data leakage risks
- Unsafe tool invocation
-

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

- Prompt injection via user or document input
- Sensitive data leakage risks
- Unsafe tool invocation
- Unvalidated outputs reaching users

THE PROBLEM WE'RE SOLVING

AI features are vulnerable at multiple trust boundaries.

- Prompt injection via user or document input
- Sensitive data leakage risks
- Unsafe tool invocation
- Unvalidated outputs reaching users

Now: apply layered controls and safe fallback paths.

WHY THIS MATTERS IN DELIVERY

-
-
-
-

WHY THIS MATTERS IN DELIVERY

- Security failures are usually architecture failures, not prompt mistakes.
-
-
-

WHY THIS MATTERS IN DELIVERY

- Security failures are usually architecture failures, not prompt mistakes.
- Most incidents happen at trust boundaries.
-
-

WHY THIS MATTERS IN DELIVERY

- Security failures are usually architecture failures, not prompt mistakes.
- Most incidents happen at trust boundaries.
- Guardrails must be engineered into the workflow, not bolted on later.
-

WHY THIS MATTERS IN DELIVERY

- Security failures are usually architecture failures, not prompt mistakes.
- Most incidents happen at trust boundaries.
- Guardrails must be engineered into the workflow, not bolted on later.
- Safe fallback is part of product quality in regulated environments.

GUARDRAIL LAYERS

-
-
-
-
-

GUARDRAIL LAYERS

- Input screening and trust labelling
-
-
-
-

GUARDRAIL LAYERS

- Input screening and trust labelling
- Prompt constraints and system boundaries
-
-
-

GUARDRAIL LAYERS

- Input screening and trust labelling
- Prompt constraints and system boundaries
- Tool permission and parameter constraints
-
-

GUARDRAIL LAYERS

- Input screening and trust labelling
- Prompt constraints and system boundaries
- Tool permission and parameter constraints
- Output validation and refusal handling
-

GUARDRAIL LAYERS

- Input screening and trust labelling
- Prompt constraints and system boundaries
- Tool permission and parameter constraints
- Output validation and refusal handling
- Fallback behaviour for uncertain responses

TRUST BOUNDARY MAP

Boundary	Typical examples	Default level
User input	Free text, form fields, uploads	Untrusted
Document / retrieved text	OCR, retrieved chunks, attachments	Untrusted
Tool responses	External APIs, search tools	Semi-trusted
Internal policy / config	Allowlists, schemas, thresholds	Trusted

*Rule: assume untrusted unless explicitly
verified otherwise.*

PROMPT INJECTION: HOW IT ENTERS SYSTEMS

-
-
-
-

PROMPT INJECTION: HOW IT ENTERS SYSTEMS

- Direct user messages that attempt to override instructions.
-
-
-

PROMPT INJECTION: HOW IT ENTERS SYSTEMS

- Direct user messages that attempt to override instructions.
- Embedded instructions inside uploaded documents.
-
-

PROMPT INJECTION: HOW IT ENTERS SYSTEMS

- Direct user messages that attempt to override instructions.
- Embedded instructions inside uploaded documents.
- Retrieved context that contains adversarial content.
-

PROMPT INJECTION: HOW IT ENTERS SYSTEMS

- Direct user messages that attempt to override instructions.
- Embedded instructions inside uploaded documents.
- Retrieved context that contains adversarial content.
- Tool output treated as authority without validation.

PROMPT INJECTION: DEFENCE PATTERNS

-
-
-
-

PROMPT INJECTION: DEFENCE PATTERNS

- Separate instructions from user and document content structurally.
-
-
-

PROMPT INJECTION: DEFENCE PATTERNS

- Separate instructions from user and document content structurally.
- Use an explicit instruction hierarchy: system over policy over user.
-
-

PROMPT INJECTION: DEFENCE PATTERNS

- Separate instructions from user and document content structurally.
- Use an explicit instruction hierarchy: system over policy over user.
- Treat all retrieved and document content as data only, never as instructions.
-

PROMPT INJECTION: DEFENCE PATTERNS

- Separate instructions from user and document content structurally.
- Use an explicit instruction hierarchy: system over policy over user.
- Treat all retrieved and document content as data only, never as instructions.
- Reject or sanitise known hostile patterns before prompt assembly.

INPUT GUARDRAILS (PRE-CALL)

-
-
-
-

INPUT GUARDRAILS (PRE-CALL)

- Validate payload schema and required fields.
-
-
-

INPUT GUARDRAILS (PRE-CALL)

- Validate payload schema and required fields.
- Enforce size, type, and format limits.
-
-

INPUT GUARDRAILS (PRE-CALL)

- Validate payload schema and required fields.
- Enforce size, type, and format limits.
- Classify and label the trust level of each input source.
-

INPUT GUARDRAILS (PRE-CALL)

- Validate payload schema and required fields.
- Enforce size, type, and format limits.
- Classify and label the trust level of each input source.
- Apply PII and sensitive-content detection where required.

INPUT GUARDRAILS (PRE-CALL)

- Validate payload schema and required fields.
- Enforce size, type, and format limits.
- Classify and label the trust level of each input source.
- Apply PII and sensitive-content detection where required.

Failing pre-call is cheap. Failing post-call is not.

TOOL GUARDRAILS

-
-
-
-

TOOL GUARDRAILS

- Allowlist tool access per workflow, not per request.
-
-
-

TOOL GUARDRAILS

- Allowlist tool access per workflow, not per request.
- Constrain tool parameters and output size explicitly.
-
-

TOOL GUARDRAILS

- Allowlist tool access per workflow, not per request.
- Constrain tool parameters and output size explicitly.
- Set timeout and retry limits per tool.
-

TOOL GUARDRAILS

- Allowlist tool access per workflow, not per request.
- Constrain tool parameters and output size explicitly.
- Set timeout and retry limits per tool.
- Require approval gates for high-impact actions.

TOOL GUARDRAILS

- Allowlist tool access per workflow, not per request.
- Constrain tool parameters and output size explicitly.
- Set timeout and retry limits per tool.
- Require approval gates for high-impact actions.

Internal tools still need constraints and observability.

OUTPUT GUARDRAILS (POST-CALL)

-
-
-
-

OUTPUT GUARDRAILS (POST-CALL)

- Schema validation for contract correctness.
-
-
-

OUTPUT GUARDRAILS (POST-CALL)

- Schema validation for contract correctness.
- Policy validation for business and compliance rules.
-
-

OUTPUT GUARDRAILS (POST-CALL)

- Schema validation for contract correctness.
- Policy validation for business and compliance rules.
- Confidence and uncertainty thresholds for acceptance.
-

OUTPUT GUARDRAILS (POST-CALL)

- Schema validation for contract correctness.
- Policy validation for business and compliance rules.
- Confidence and uncertainty thresholds for acceptance.
- Redaction before returning output to the frontend.

OUTPUT GUARDRAILS (POST-CALL)

- Schema validation for contract correctness.
- Policy validation for business and compliance rules.
- Confidence and uncertainty thresholds for acceptance.
- Redaction before returning output to the frontend.

Schema passing is not the same as policy passing. Both are required.

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

-
-
-
-

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

- Refuse unsafe requests with a clear, consistent explanation.
-
-
-

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

- Refuse unsafe requests with a clear, consistent explanation.
- Return a deterministic `needs_review` shape for ambiguous outputs.
-
-

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

- Refuse unsafe requests with a clear, consistent explanation.
- Return a deterministic `needs_review` shape for ambiguous outputs.
- Preserve the response contract so the frontend does not need special-case handling.
-

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

- Refuse unsafe requests with a clear, consistent explanation.
- Return a deterministic `needs_review` shape for ambiguous outputs.
- Preserve the response contract so the frontend does not need special-case handling.
- Log the reason and trace for follow-up and audit.

REFUSAL AND FALLBACK DESIGN

When a response is uncertain or non-compliant:

- Refuse unsafe requests with a clear, consistent explanation.
- Return a deterministic `needs_review` shape for ambiguous outputs.
- Preserve the response contract so the frontend does not need special-case handling.
- Log the reason and trace for follow-up and audit.

Refusal and needs-review are safe product behaviours, not error states.

WHAT NOT TO DO

-
-
-
-

WHAT NOT TO DO

- Apply one regex and call it prompt injection protection.
-
-
-

WHAT NOT TO DO

- Apply one regex and call it prompt injection protection.
- Trust model output because the schema parsed once.
-
-

WHAT NOT TO DO

- Apply one regex and call it prompt injection protection.
- Trust model output because the schema parsed once.
- Allow unrestricted tool invocation.
-

WHAT NOT TO DO

- Apply one regex and call it prompt injection protection.
- Trust model output because the schema parsed once.
- Allow unrestricted tool invocation.
- Expose raw model output directly to end users.

SECURITY TEST SCENARIOS

- 1.
- 2.
- 3.
- 4.
- 5.

SECURITY TEST SCENARIOS

1. Injection string embedded in a document body.
- 2.
- 3.
- 4.
- 5.

SECURITY TEST SCENARIOS

1. Injection string embedded in a document body.
2. Oversized input payload designed to stress limits.
- 3.
- 4.
- 5.

SECURITY TEST SCENARIOS

1. Injection string embedded in a document body.
2. Oversized input payload designed to stress limits.
3. Tool response with unexpected or malformed structure.
- 4.
- 5.

SECURITY TEST SCENARIOS

1. Injection string embedded in a document body.
2. Oversized input payload designed to stress limits.
3. Tool response with unexpected or malformed structure.
4. Output that passes schema but fails policy validation.
- 5.

SECURITY TEST SCENARIOS

1. Injection string embedded in a document body.
2. Oversized input payload designed to stress limits.
3. Tool response with unexpected or malformed structure.
4. Output that passes schema but fails policy validation.
5. Low-confidence extraction that should trigger fallback.

GOVERNANCE AND REGIONAL CONTEXT

-
-
-
-

GOVERNANCE AND REGIONAL CONTEXT

- Controls should produce audit evidence by default, not on request.
-
-
-

GOVERNANCE AND REGIONAL CONTEXT

- Controls should produce audit evidence by default, not on request.
- Trace retention and response explainability are operational requirements.
-
-

GOVERNANCE AND REGIONAL CONTEXT

- Controls should produce audit evidence by default, not on request.
- Trace retention and response explainability are operational requirements.
- Regional deployment constraints (EU/US) may affect tooling and data routing paths.
-

GOVERNANCE AND REGIONAL CONTEXT

- Controls should produce audit evidence by default, not on request.
- Trace retention and response explainability are operational requirements.
- Regional deployment constraints (EU/US) may affect tooling and data routing paths.
- Guardrails should be reusable patterns shared across features, not per-feature improvisations.

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

-
-
-
-
-

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
-
-
-
-

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
- Post-call schema and policy checks
-
-
-

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
- Post-call schema and policy checks
- Tool constraints and approval gates
-
-

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
- Post-call schema and policy checks
- Tool constraints and approval gates
- Refusal and fallback behaviour with deterministic response shape
-

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
- Post-call schema and policy checks
- Tool constraints and approval gates
- Refusal and fallback behaviour with deterministic response shape
- Trace fields covering security-relevant decisions

MODULE 4 LAB BUILD TARGET

You will harden an existing workflow by adding:

- Input screening and trust labelling
- Post-call schema and policy checks
- Tool constraints and approval gates
- Refusal and fallback behaviour with deterministic response shape
- Trace fields covering security-relevant decisions

Definition of done: every trust boundary has an explicit control and the fallback path is tested.

SUMMARY

- 1.
- 2.
- 3.
- 4.

SUMMARY

1. **Security is boundary design plus control-flow discipline.**
- 2.
- 3.
- 4.

SUMMARY

1. **Security is boundary design** plus control-flow discipline.
2. **Guardrails are layered:** input, prompt, tool, output, fallback.
- 3.
- 4.

SUMMARY

1. **Security** is boundary design plus control-flow discipline.
2. **Guardrails** are layered: input, prompt, tool, output, fallback.
3. **Validation** is dual: schema correctness and policy compliance.
- 4.

SUMMARY

1. **Security** is boundary design plus control-flow discipline.
2. **Guardrails** are layered: input, prompt, tool, output, fallback.
3. **Validation** is dual: schema correctness and policy compliance.
4. **Safe fallback** is a success path, not an exception path.

BRIDGE TO MODULE 5

What we have now:

-

What is next:

-

BRIDGE TO MODULE 5

What we have now:

- A hardened backend workflow with layered controls at every trust boundary.

What is next:

-

BRIDGE TO MODULE 5

What we have now:

- A hardened backend workflow with layered controls at every trust boundary.

What is next:

- Secure backend outputs still need responsible UX to be useful.

BRIDGE TO MODULE 5

What we have now:

- A hardened backend workflow with layered controls at every trust boundary.

What is next:

- Secure backend outputs still need responsible UX to be useful.

Module 5 covers frontend AI UX patterns, including how to present uncertainty clearly without undermining user trust.

QUESTIONS?

Module 4 — Governed AI Feature Delivery